Aula 16 – Strings em C: Introdução e Aplicações

1. Objetivo geral:

Compreender como a linguagem C representa e manipula textos por meio de vetores de caracteres, desenvolvendo programas que realizem leitura, armazenamento e processamento de strings.

2. Relembrando....

Nas aulas anteriores, aprendemos a utilizar vetores e matrizes para armazenar conjuntos de valores.

Por exemplo:

```
int idades[3] = {18, 20, 22};

float notas[2][3] = {
    {7.0, 8.0, 9.0},
    {6.0, 7.5, 8.5}
};
```

Mas como armazenar informações como nomes de alunos, cidades, produtos e mensagens?

Uma variável char armazena um único caractere:

```
char letra = 'A';
```

Entretanto, um nome como "Maria" possui vários caracteres. É nesse momento que introduzimos as strings.

3. Situação-problema: Cadastro de alunos

Imagine que precisamos desenvolver um sistema para cadastrar o nome e a idade de um aluno. O sistema deverá receber essas informações e apresentá-las na tela. Qual tipo de dado devemos utilizar para armazenar o nome completo do aluno?

Em C, podemos utilizar um vetor de caracteres.

Exemplo 1:

```
char nome[50];
int idade;
```

A variável **nome** reserva espaço para até 49 caracteres de texto e o caractere especial de encerramento '\0'.

Exemplo 2:

```
#include <stdio.h>

int main(void) {

    char nome[50];

    int idade;

    printf("Digite seu nome completo: ");

    fgets(nome, sizeof(nome), stdin);

    printf("Digite sua idade: ");

    scanf("%d", &idade);

    printf("\nNome: %s", nome);

    printf("Idade: %d\n", idade);

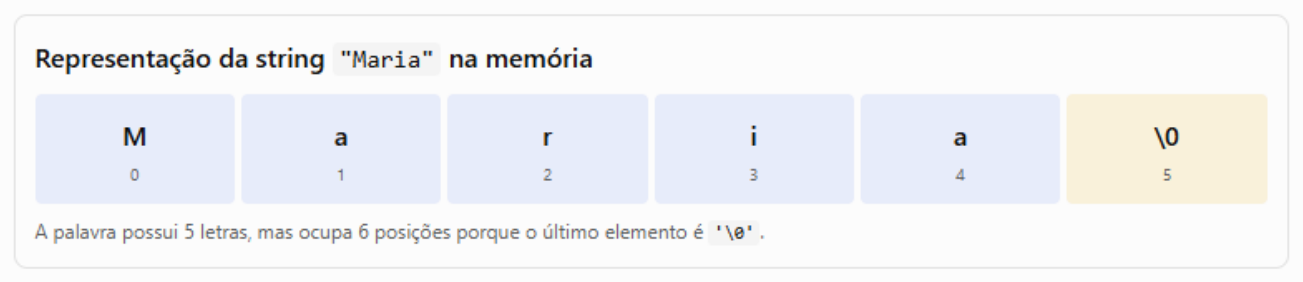
    return 0;

}
```

4. O que é uma string em C?

Uma string é uma sequência de caracteres armazenada em um vetor do tipo char, finalizada pelo caractere nulo '\0'. Por exemplo:

```
char nome[6] = "Maria";
```



Diferença importante:

```
char letra = 'M';       // Um caractere
char nome[] = "Maria"; // Uma string
```

Em C, utilizamos aspas simples para representar um caractere e aspas duplas para representar uma string literal.

5. Lendo e exibindo strings

Exemplo 1 – Utilizando scanf

```
#include <stdio.h>

int main(void) {
    char nome[30];

    printf("Digite seu primeiro nome: ");
    scanf("%29s", nome);

    printf("Ola, %s!\n", nome);

    return 0;
}
```

Neste caso, %s é utilizado para ler e exibir strings (sequências de caracteres). O scanf() com %s interrompe a leitura ao encontrar o primeiro espaço em branco.

O limite %29s evita que sejam lidos mais de 29 caracteres, preservando uma posição no vetor para o caractere terminador '\0', que indica o fim da string.

Exemplo: se eu digitar meu nome e sobrenome separados por um espaço, o programa exibirá apenas o primeiro nome, pois o scanf() com %s interrompe a leitura ao encontrar o espaço em branco.

Por que não utilizamos & em scanf("%29s", nome)? Na leitura de uma string armazenada em um vetor de caracteres, utilizamos o nome do vetor sem &, pois ele já fornece o endereço de seu primeiro elemento nessa expressão.

```
int idade;

char nome[30];
```

```
scanf("%d", &idade);

scanf("%29s", nome);
```

Exemplo 2 – Lendo nomes completos com fgets

```
#include <stdio.h>

int main(void) {
    char nome[50];

    printf("Digite seu nome completo: ");
    fgets(nome, sizeof(nome), stdin);

    printf("Nome informado: %s", nome);

    return 0;
}
```

fgets permite ler textos contendo espaços e limita a quantidade de caracteres armazenados no vetor.

Observação: fgets pode manter o caractere de quebra de linha '\n' quando há espaço disponível no vetor. Na próxima aula, poderemos aprender como removê-lo.

Para resolver o problema do exemplo 1, podemos utilizar fgets() para ler o nome completo ou uma sequência de caracteres com espaços, sem interromper a leitura ao encontrar um espaço em branco.

6. Acessando os caracteres de uma string

Assim como nos vetores, podemos acessar individualmente os caracteres de uma string por meio de seus índices.

```
#include <stdio.h>

int main(void) {
    char palavra[] = "CASA";

    printf("%c\n", palavra[0]);
    printf("%c\n", palavra[1]);
    printf("%c\n", palavra[2]);
```

```
    printf("%c\n", palavra[3]);

    return 0;
}
```

Saída esperada:

```
C
A
S
A
```

Exemplo com repetição:

```
#include <stdio.h>

int main(void) {
    char palavra[] = "CASA";

    for (int i = 0; palavra[i] != '\0'; i++) {
        printf("%c\n", palavra[i]);
    }

    return 0;
}
```

Como funciona?

- i = 0: inicia a leitura pelo primeiro caractere.
- palavra[i] != '\0': verifica se ainda não chegou ao final da string.
- i++: avança para o próximo caractere.
- printf("%c", palavra[i]): exibe o caractere atual.

O laço termina quando encontra '\0', evitando que continue percorrendo posições além do texto.

7. Modificando caracteres de uma string

Além de acessar os caracteres, também podemos modificá-los individualmente.

```
#include <stdio.h>

int main(void) {
    char palavra[] = "CASA";
```

```
    palavra[0] = 'M';

    printf("%s\n", palavra);

    return 0;
}
```

Uma string armazenada em um vetor modificável permite alterar caracteres individualmente por meio de seus índices.

Atividade para entrega:

Desafio – Contador de caracteres:

1. Uma aplicação precisa identificar quantos caracteres foram digitados pelo usuário em uma palavra. Desenvolva um programa em C que receba uma palavra, percorra seus caracteres utilizando for e apresente a quantidade de letras digitadas. Nesta atividade, você deverá realizar a contagem sem utilizar strlen, pois essa função será estudada na próxima aula.

Exercícios de fixação

2. Ler e exibir o nome completo de uma pessoa.

3. Ler uma palavra e apresentar somente seu primeiro caractere.

4. Ler uma palavra e exibir cada caractere em uma linha.

5. Ler uma palavra e contar quantas vogais ela possui, considerando letras minúsculas.

Exercícios de revisão de conteúdo:

6. Qual é a diferença entre char e uma string?

7. Por que uma string precisa do caractere '\0'?

8. Qual é a diferença entre ler um texto com scanf("%s", ...) e com fgets(...)?